Chapitre 0 — Avant tout | Partie D — Des formes | leçon 0.63.2 sur 153 | difficulté 1 / 10

0.63.2. Trois formes ensemble — doublé, deux positions

Les trois formes du 0.63, puis la spirale du D.

1. Ce qu'il faut comprendre

C'est le 0.63 et le 0.63.1 réunis : quatre formes, dont deux spirales.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <losange>` → 0.56.3. La fonction losange() — tourner en losange
- `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police
- `#include <rectangle>` → 0.57.3. La fonction rectangle() — tourner en rectangle
- `#include <spirale>` → 0.58.3. La fonction spirale() — tourner en spirale

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`#include <losange>  // une tuile qui tourne en losange`	Demande à la console « losange » (ce que c'est : une tuile qui tourne en losange). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « une tuile qui tourne en losange ».)
`#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».)
`#include <rectangle>  // une tuile qui tourne en rectangle`	Demande à la console « rectangle » (ce que c'est : une tuile qui tourne en rectangle). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « une tuile qui tourne en rectangle ».)
`#include <spirale>  // une tuile qui tourne en spirale`	Demande à la console « spirale » (ce que c'est : une tuile qui tourne en spirale). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « une tuile qui tourne en spirale ».)

`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`losange(4, 4, ALPHABET[0], 2, 1, 100, 1);` `// le A, en haut à gauche`	Appel de la fonction losange de la console, avec les arguments 4, 4, ALPHABET[0], 2, 1, 100, 1. Ce qu'elle fait : le carré posé sur la pointe, ses côtés en diagonale, autour de (x, y) : les pointes sont à « taille » cases du centre. Mêmes réglages que carre() ; accepte aussi un Carre : losange(ronde);. (Le commentaire le dit : « le A, en haut à gauche ».)
`rectangle(14, 4, ALPHABET[1], 4, 2, -1, 100, 1);` `// le B, en haut à droite`	Appel de la fonction rectangle de la console, avec les arguments 14, 4, ALPHABET[1], 4, 2, -1, 100, 1. Ce qu'elle fait : un rectangle autour de (x, y) : largeur 4 et hauteur 2 font 9 × 5 cases (2 × n + 1 chacune). sens 1 ou -1, vitesse en ms écrite en clair, tours. Largeur jusqu'à 9, hauteur jusqu'à 8. (Le commentaire le dit : « le B, en haut à droite ».)
`spirale(4, 12, ALPHABET[2], 2, 1, 100);` `// le C, en bas à gauche`	Appel de la fonction spirale de la console, avec les arguments 4, 12, ALPHABET[2], 2, 1, 100. Ce qu'elle fait : part du centre et tourne en s'éloignant (branches de 1, 1, 2, 2, 3, 3… pas) jusqu'au bord d'un carré de « taille », puis revient au centre. sens 1 ou -1, vitesse en ms écrite en clair. (Le commentaire le dit : « le C, en bas à gauche ».)
`spirale(14, 12, ALPHABET[3], 2, 1, 100);` `// le D, en bas à droite`	Appel de la fonction spirale de la console, avec les arguments 14, 12, ALPHABET[3], 2, 1, 100. Ce qu'elle fait : part du centre et tourne en s'éloignant (branches de 1, 1, 2, 2, 3, 3… pas) jusqu'au bord d'un carré de « taille », puis revient au centre. sens 1 ou -1, vitesse en ms écrite en clair. (Le commentaire le dit : « le D, en bas à droite ».)
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2416 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        49 o  ← la console, à l'allumage  — la mise en route de la console
  losange()       449 o  ← main()  — une fonction fournie par le langage
  rectangle()     527 o  ← main()  — une fonction fournie par le langage
  spirale()       442 o  ← main()  — une fonction fournie par le langage
  main()          127 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces    12 o  ← EffacerCases  — attend de pouvoir écrire une case
  AdresseCase      10 o  ← poser(), effacer()  — trouve l'adresse d'une case de l'écran
  EffacerCases     12 o  ← effacer()  — efface des cases
  CopierTuiles     18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles          704 o  ← CopierTuiles  — la police entière
```

```
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier,
DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette,
DonneesTransfert, Notes
Fonctions du langage appelées :
  poser()     × 11   143 o sur place, dans losange(), rectangle(), spirale()  + AdresseCase (10 o)
  effacer()   ×  8   120 o sur place, dans losange(), rectangle(), spirale()  + AdresseCase (10 o), EffacerCases (12
o)
  image()     ×  9    27 o sur place, dans losange(), rectangle(), spirale(), main()  + AttendreImage (28 o)
Tes fonctions appelées :
  spirale()   ×  2    54 o sur place, dans main()
  rectangle() ×  1    35 o sur place, dans main()
  losange()   ×  1    31 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <losange>     // une tuile qui tourne en losange
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
#include <rectangle>   // une tuile qui tourne en rectangle
#include <spirale>     // une tuile qui tourne en spirale
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

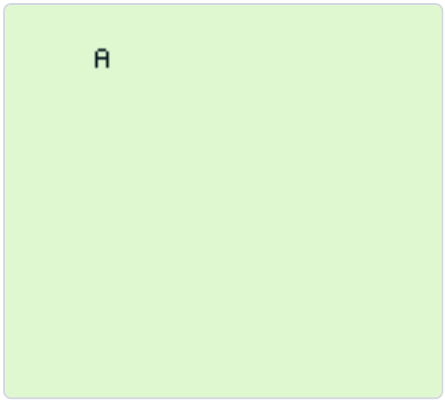
```
int main() {
  losange(4, 4, ALPHABET[0], 2, 1, 100, 1);            // le A, en haut à gauche
  rectangle(14, 4, ALPHABET[1], 4, 2, -1, 100, 1);    // le B, en haut à droite
  spirale(4, 12, ALPHABET[2], 2, 1, 100);             // le C, en bas à gauche
  spirale(14, 12, ALPHABET[3], 2, 1, 100);            // le D, en bas à droite

  while (true) {
    image();
  }
}
```

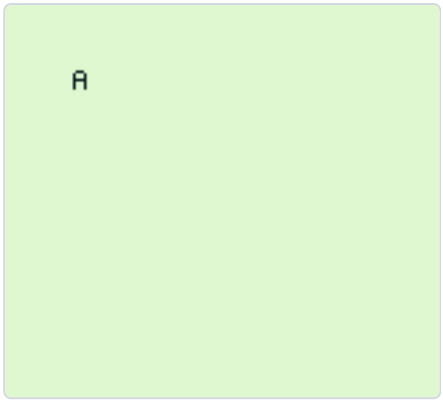
3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

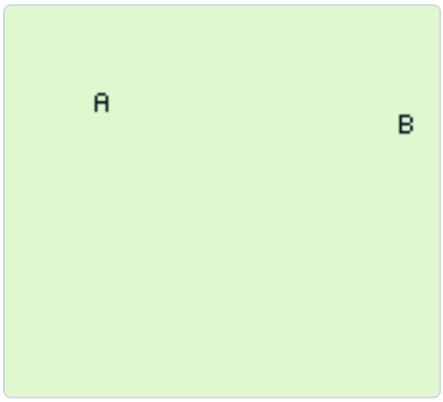
la toute première image — la console vient de démarrer — image 1



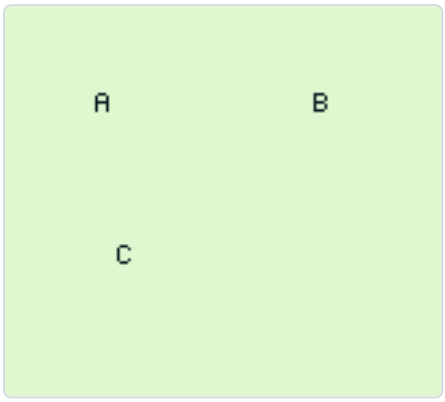
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

> Losange, rectangle, spirale, puis une seconde spirale en bas à droite.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le C finit en (4, 12) (4,12)

✓ le D finit en (14, 12) (14,12)

Ensuite : **0.64. Quatre formes ensemble** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++